Phase 1 Capstone Report

Student Grade Analyser with Pattern Matching

BACHELOR OF TECHNOLOGY (CSE/ AI&ML)

Semester-VIII

to: Hemraj Kumawat

ADITI GOYAL (2301730298) **Section-E**

JHALAK DABAS (2301730322) **Section-E**

Department of Computer Science and Engineering

School of Engineering and Technology

1. Problem Statement & Motivation

1.1 Problem Description

After an examination, a teacher receives a list of student marks and must turn it into useful information: the grade of each student, the class average, the highest scorer, and how many students passed or failed. In many classrooms this is still done manually or with simple spreadsheet formulas, which is time-consuming and can lead to mistakes such as applying a wrong grade boundary or miscounting the number of failures.

Grading is based on fixed rules (for example, "90 and above is A+, below 40 is F"). When such rules are written as long chains of nested if-else statements, the code becomes difficult to read and easy to get wrong. A clear and simple way of expressing these rules is needed.

This project builds a small console application that stores student marks, assigns grades using clearly written rules, and shows the class average, the topper and the pass/fail statistics through a simple menu.

1.2 Real-World Context

The project belongs to the domain of a CLI (command-line) data-processing utility for academic result analysis. It can be used by teachers or students for a quick analysis of a class result, and it runs on any computer without a database or internet connection.

1.3 Motivation

- It removes manual calculation errors by applying one grading rule to every student.
- It gives results instantly through a simple menu.
- It is a good practical application of functional programming concepts from ENSP465: lists, pure functions, pattern matching and higher-order functions.

1.4 Existing Solutions and Their Limitations

Spreadsheets (Excel, Google Sheets) can calculate averages and grades, but multi-level grading needs long nested IF formulas that are hard to read and check. Institutional ERP result modules are heavy, need login and server access, and are not suitable for quick offline use. The proposed tool is lightweight and its grading rules are written as short, readable pattern-matching cases.

2. Proposed Solution & Language Justification

2.1 Project Overview

The Student Grade Analyser is an F# console-based application that analyses student marks and assigns grades using pattern matching. It calculates the class average, identifies the topper, and

displays pass/fail statistics using lists, functions and higher-order functions, through a simple menu-driven interface.

2.2 Language Chosen

F# (a functional-first language on the .NET platform) is the single implementation language.

2.3 Language Justification

1. **Pattern matching with guards.** The grade boundaries are written as a short list of match cases (| m when m >= 90.0 -> "A+"). This is easier to read than nested if-else and the compiler warns if a case is left unhandled.
2. **Immutable records and lists.** Student data is stored in an immutable list of records (Name, Marks), so the data cannot be changed accidentally while it is being analysed.
3. **Higher-order functions and the pipeline operator.** Statistics are computed directly with List.averageBy, List.maxBy, List.filter and List.iter, joined by the |> operator. Each calculation is a single readable line, without manual loops or counters.
4. **Type inference and concise syntax.** F# is statically typed but infers most types, so the whole program stays short while errors are still caught at compile time.

2.4 Libraries and Tools

Library / Tool	Purpose
FSharp.Core	List module functions, records, pattern matching and the pipeline operator
System (.NET base library)	Console.ReadLine for reading the menu choice
.NET SDK 8.0 or later	Compiling and running the program with the dotnet CLI
Visual Studio Code with Ionide	Code editor for F#

3. Objectives & Scope

3.1 Objectives

1. Implement a getGrade function using F# pattern matching that returns the correct grade (A+, A, B, C, D or F) for every mark, including the boundary values 40, 60, 70, 80 and 90.
2. Store the details of at least 5 students in an immutable list of records and display each student's name, marks, grade and Pass/Fail result in a table.
3. Calculate the class average and identify the topper using List.averageBy and List.maxBy.
4. Count the number of students who passed and failed (pass mark = 40) using List.filter.

5. Provide a menu-driven console interface with 5 options that handles an invalid choice without crashing.

3.2 In-Scope Features

- Menu: display all students, class average, topper, pass/fail count, exit.
- Grade assignment by pattern matching.
- Class average, topper and pass/fail statistics.

3.3 Out-of-Scope

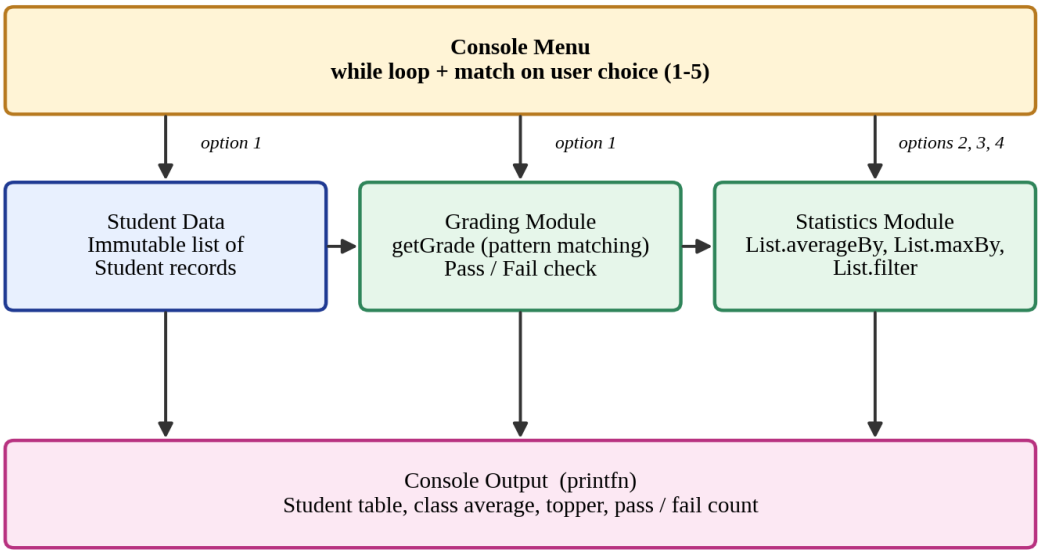
- No graphical user interface (GUI), web or mobile version.
- No database, file handling, or login system.
- No multi-subject marks or GPA/CGPA calculation.

4. System Architecture & Design

4.1 High-Level Architecture

The application has a simple structure. A console menu (a while loop with a match expression) receives the user's choice and calls the required function. The student data is held in an immutable list. The grading and statistics functions read this list and the results are printed to the console.

Figure 1: High-Level Architecture of the Student Grade Analyser



4.2 Component Description

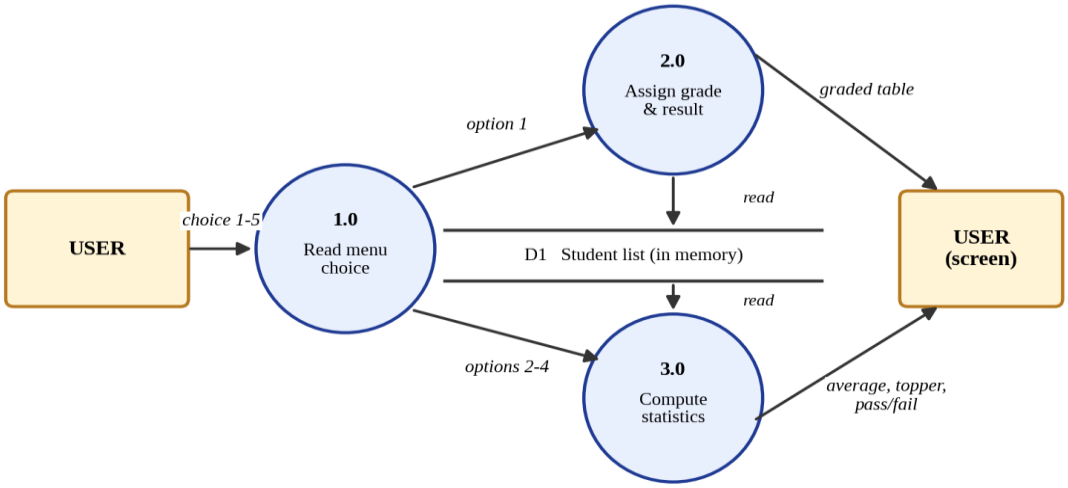
Component	Description	Input	Output
Student record	Record type with Name (string) and Marks (float).	—	Student value

Component	Description	Input	Output
students list	Immutable list holding the class data.	—	List of Student
getGrade	Pattern-matching function that converts marks into a grade.	Marks (float)	Grade (string)
showStudents	Prints a table of name, marks, grade and Pass/Fail result.	Student list	Table on console
average / topper	Compute the class average and highest scorer using List.averageBy and List.maxBy.	Student list	Number / Student
passed / failed	Filter students with marks ≥ 40 or < 40.	Student list	Two lists (counts shown)
Menu loop	Shows options, reads the choice and calls the matching function.	Choice 1–5	Calls the functions

4.3 Data Flow Diagram

Figure 2 shows the Level 1 data flow diagram. The user gives a menu choice; Process 1.0 reads it and passes control to Process 2.0 (grading and result display) or Process 3.0 (statistics). Both processes read the student list (D1) and send their output to the screen.

Figure 2: Data Flow Diagram (Level 1) - Student Grade Analyser



Legend: circle = process, rectangle = external entity, parallel lines = data store, arrow = data flow

4.4 Input–Process–Output (IPO)

Stage	Description
Input	Menu choice (1–5) typed by the user. Student names and marks are stored in the program as a list.
Process	Convert marks into grades with pattern matching; decide Pass/Fail (≥ 40); calculate the class average with List.averageBy; find the topper with List.maxBy; separate passed and failed students with List.filter.
Output	Table of Name, Marks, Grade and Result; class average; topper's name and marks; number of students passed and failed.

4.5 Key Data Structures and F# Features

F# Feature	Use in this project
Record (Student)	Holds a student's name and marks as one immutable value.
List	Stores all students; used with List.map-style processing.
Pattern matching with guards	Grade assignment (getGrade) and the menu choice.
Higher-order functions	List.iter, List.averageBy, List.maxBy, List.filter.
Pipeline operator (|>)	Passes the list through each function step by step.

Planned grading rule (design sketch):

```
type Student = { Name: string; Marks: float }

let getGrade marks =
    match marks with
    | m when m >= 90.0 -> "A+"
    | m when m >= 80.0 -> "A"
    | m when m >= 70.0 -> "B"
    | m when m >= 60.0 -> "C"
    | m when m >= 40.0 -> "D"
    | _                -> "F"
```

Planned grading scale:

Marks	Grade	Result
90 and above	A+	Pass
80 – 89	A	Pass
70 – 79	B	Pass
60 – 69	C	Pass
40 – 59	D	Pass

Marks	Grade	Result
Below 40	F	Fail

5. Work Plan & Timeline

5.1 Phase II Deliverables (Implementation)

- Complete working program with all five menu options.
- Demonstration of grading and statistics on the sample student data.
- Changes based on faculty feedback on Phase I.

5.2 Phase III Deliverables (Testing, Documentation and Viva)

- Testing of grade boundaries and the invalid-menu-choice case.
- Final source code and project report.
- Viva preparation on pattern matching, lists and higher-order functions.

5.3 Task Distribution

Individual project - all modules are developed, tested and documented by the submitting student.

6. References

References are in IEEE format. Online sources were last accessed in October 2026.

[1] Microsoft, “F# documentation,” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/fsharp/ . Accessed: October 2026.

[2] Microsoft, “Match expressions (F# language reference),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/match-expressions . Accessed: October 2026.

[3] Microsoft, “Lists (F# language reference),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/lists . Accessed: October 2026.

[4] D. Syme, A. Granicz, and A. Cisternino, Expert F# 4.0, 4th ed. New York, NY, USA: Apress, 2015.

[5] S. Wlaschin, “F# for Fun and Profit.” [Online]. Available: https://fsharpforfunandprofit.com/ . Accessed: October 2026.